

ARM PrimeCell™

Vectored Interrupt Controller (PL190)

Technical Reference Manual



Vectored Interrupt Controller (PL190)

Technical Reference Manual

Copyright © 2000 ARM Limited. All rights reserved.

Release Information

The following changes have been made to this document.

Change history

Date	Issue	Change
30 June 2000	A	First release
August 2000	B	Small corrections to code examples.
September 2000	C	VICITIP1 & 2 changed to read only. Changes to Figs 2-5 & 2-6.

Proprietary Notice

ARM, the ARM Powered logo, Thumb, and StrongARM are registered trademarks of ARM Limited.

The ARM logo, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, PrimeCell, ARM7TDMI, ARM7TDMI-S, ARM9TDMI, ARM9E-S, ARM966E-S, ETM7, ETM9, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM in good faith. However, all warranties implied or expressed, including but not limited to implied warranties of merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Confidentiality Status

This document is Open Access. This document has no restriction on distribution.

Product Status

The information in this document is final (information on a developed product).

Web Address

<http://www.arm.com>

Contents

ARM PrimeCell Vectored Interrupt Controller (PL190) Technical Reference Manual

Preface

About this document	iv
Further reading	vi
Feedback	vii

Chapter 1

Introduction

1.1 About the ARM PrimeCell Vectored Interrupt Controller (PL190)	1-2
---	-----

Chapter 2

Functional Overview

2.1 ARM PrimeCell Vectored Interrupt Controller (PL190) overview	2-2
2.2 PrimeCell VIC operation	2-10
2.3 PrimeCell VIC connectivity	2-12

Chapter 3

Programmer's Model

3.1 About the programmer's model	3-2
3.2 Summary of PrimeCell VIC registers	3-3
3.3 Register descriptions	3-6
3.4 Interrupt latency	3-19
3.5 Interrupt priority	3-22

Chapter 4	Programmer's Model for Test	
4.1	PrimeCell VIC test harness overview	4-2
4.2	Scan testing	4-3
4.3	Test registers	4-4
4.4	Integration tests	4-7
4.5	Integration test summary	4-11
Appendix A	ARM PrimeCell Vectored Interrupt Controller (PL190) Signal Descriptions	
A.1	AMBA AHB signals	A-2
A.2	Interrupt controller signals	A-4
A.3	Daisy chain signals	A-5
A.4	Scan test control signals	A-6
Appendix B	ARM PrimeCell Vectored Interrupt Controller (PL190) Example Code	
B.1	About the example code	B-2

Preface

This preface introduces the ARM PrimeCell Vectored Interrupt Controller (PL190) and its reference documentation. It contains the following sections:

- *About this document* on page iv
- *Further reading* on page vi
- *Feedback* on page vii.

About this document

This document is the technical reference manual for the ARM PrimeCell *Vectored Interrupt Controller* (VIC).

Intended audience

This document has been written for hardware and software engineers implementing System-on-Chip designs. It provides the necessary information to enable designers to integrate the peripheral into a target system as quickly as possible.

Organization

This document is organized as follows:

Chapter 1 *Introduction*

Read this chapter for an introduction to the PrimeCell VIC and its features.

Chapter 2 *Functional Overview*

Read this chapter for a description of the major functional blocks of the PrimeCell VIC.

Chapter 3 *Programmer's Model*

Read this chapter for a description of the registers and programming details of the PrimeCell VIC.

Chapter 4 *Programmer's Model for Test*

Read this chapter for a description of the test registers and signals of the PrimeCell VIC.

Appendix A *ARM PrimeCell Vectored Interrupt Controller (PL190) Signal Descriptions*

Read this appendix for a description of the PrimeCell VIC signals.

Appendix B *ARM PrimeCell Vectored Interrupt Controller (PL190) Example Code*

Read this appendix for a description of the PrimeCell VIC example code.

Typographical conventions

The following typographical conventions are used in this book:

`typewriter` Denotes text that may be entered at the keyboard, such as commands, file and program names, and source code.

typewriter Denotes a permitted abbreviation for a command or option. The underlined text may be entered instead of the full command or option name.

typewriter italic

Denotes arguments to commands and functions where the argument is to be replaced by a specific value.

italic Highlights important notes, introduces special terminology, denotes internal cross-references, and citations.

bold Highlights interface elements, such as menu names and buttons. Also used for terms in descriptive lists, where appropriate.

`typewriter bold`

Denotes language keywords when used outside example code and ARM processor signal names.

Further reading

This section lists publications by ARM Limited.

ARM periodically provides updates and corrections to its documentation. See <http://www.arm.com> for current errata sheets and addenda.

See also the ARM Frequently Asked Questions list at:
<http://www.arm.com/DevSupp/Sales+Support/faq.html>

ARM publications

This document contains information that is specific to the ARM PrimeCell Vectored Interrupt Controller (PL190). Refer to the following documents for other relevant information:

- *AMBA Specification (Rev 2.0)* (ARM IHI 00011)

Feedback

ARM Limited welcomes feedback both on the ARM PrimeCell Vectored Interrupt Controller (PL190), and on the documentation.

Feedback on the ARM PrimeCell Vectored Interrupt Controller (PL190)

If you have any comments or suggestions about this product, please contact your supplier giving:

- the product name
- a concise explanation of your comments.

Feedback on this document

If you have any comments about this document, please send an email to errata@arm.com giving:

- the document title
- the document number
- the page number(s) to which your comments refer
- a concise explanation of your comments.

General suggestions for additions and improvements are also welcome.

Chapter 1

Introduction

This chapter introduces the ARM PrimeCell Vectored Interrupt Controller (PL190) and contains the following section:

- *About the ARM PrimeCell Vectored Interrupt Controller (PL190)* on page 1-2.

1.1 About the ARM PrimeCell Vectored Interrupt Controller (PL190)

The PrimeCell *Vectored Interrupt Controller* (VIC) is an *Advanced Microcontroller Bus Architecture* (AMBA) compliant, *System-on-Chip* (SoC) peripheral that is developed, tested, and licensed by ARM.

The PrimeCell VIC provides an interface to the interrupt system, and improves interrupt latency in two ways:

- moves the interrupt controller to the AMBA AHB bus
- provides vectored interrupt support for high-priority interrupt sources.

1.1.1 Features of the PrimeCell VIC

The PrimeCell VIC has the following features:

- compliance to the *AMBA Specification (Rev 2.0)* onwards for easy integration into *System-on-Chip* (SoC) implementation
- support for 32 standard interrupts
- support for 16 vectored IRQ interrupts
- hardware interrupt priority
- IRQ and FIQ generation
- AHB mapped for faster interrupt response
- software interrupt generation
- test registers
- raw interrupt status
- interrupt request status
- interrupt masking
- privileged mode support
- vector interrupt controller daisy chaining support.

Chapter 2

Functional Overview

This chapter describes the major functional blocks of the ARM PrimeCell Vectored Interrupt Controller (PL190) and contains the following sections:

- *ARM PrimeCell Vectored Interrupt Controller (PL190) overview* on page 2-2
- *PrimeCell VIC operation* on page 2-10
- *PrimeCell VIC connectivity* on page 2-12.

2.1 ARM PrimeCell Vectored Interrupt Controller (PL190) overview

The PrimeCell *Vectored Interrupt Controller* (VIC) provides a software interface to the interrupt system. In an ARM system, two levels of interrupt are available:

- *Fast Interrupt Request* (FIQ) for fast, low latency interrupt handling
- *Interrupt Request* (IRQ) for more general interrupts.

Only a single FIQ source at a time is generally used in a system, to provide a true low-latency interrupt. This has the following benefits:

- You can execute the interrupt service routine directly without determining the source of the interrupt.
- Interrupt latency is reduced. You can use the banked registers available for FIQ interrupts more efficiently, because a context save is not required.

There are 32 interrupt lines. The VIC uses a bit position for each different interrupt source. The software can control each request line to generate software interrupts.

There are 16 vectored interrupts. These interrupts can only generate an IRQ interrupt. The vectored and nonvectored IRQ interrupts provide an address for an *Interrupt Service Routine* (ISR). Reading from the vector interrupt address register, VICVectAddr, provides the address of the ISR, and updates the interrupt priority hardware that masks out the current and any lower priority interrupt requests. Writing to the VICVectAddr register, indicates to the interrupt priority hardware that the current interrupt is serviced, allowing lower priority interrupts to go active.

The FIQ interrupt has the highest priority, followed by interrupt vector 0 to interrupt vector 15. Nonvectored IRQ interrupts have the lowest priority. A programmed interrupt request allows you to generate an interrupt under software control. This register is typically used to downgrade an FIQ interrupt to an IRQ interrupt.

———— **Note** ————

The priority of the FIQ over IRQ is set by the ARM. The VIC can raise both an FIQ and an IRQ at the same time.

The IRQ and FIQ request logic has an asynchronous path. This allows interrupts to be asserted when the clock is disabled.

By convention, for the IRQ interrupt, bits 1 to 5 must be used as defined in Table 2-1. Bit 0 and bit 6 upwards are available for use as required. For the FIQ interrupt, the bits can be used as required.

Table 2-1 Interrupt standard configuration

Bit	Interrupt source
1	Software interrupt
2	Comms Rx
3	Comms Tx
4	Timer 1
5	Timer 2

Figure 2-1 on page 2-4 shows a block diagram of the PrimeCell VIC.

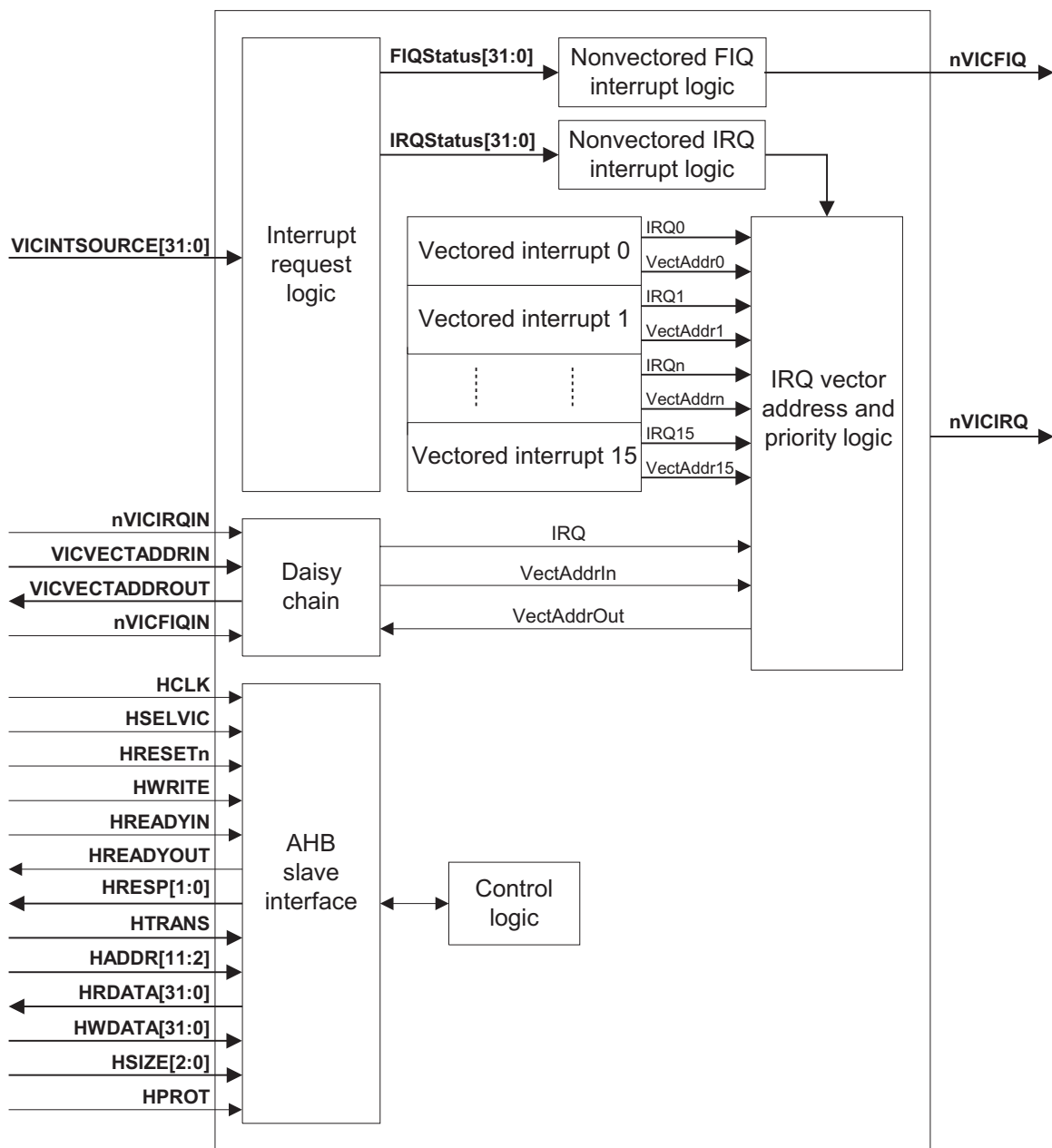


Figure 2-1 VIC block diagram

The main components of the PrimeCell VIC are described in the following sections:

- *Interrupt request logic*
- *Nonvectored FIQ interrupt logic* on page 2-6
- *Nonvectored IRQ interrupt logic* on page 2-6
- *Vectored interrupt block* on page 2-7
- *Interrupt priority logic* on page 2-8.

2.1.1 Interrupt request logic

The interrupt request logic receives the interrupt requests from the peripheral and combines them with the software interrupt requests. It then masks out the interrupt requests which are not enabled, and routes the enabled interrupt requests to either IRQ or FIQ. Figure 2-2 shows a block diagram of the interrupt request logic.

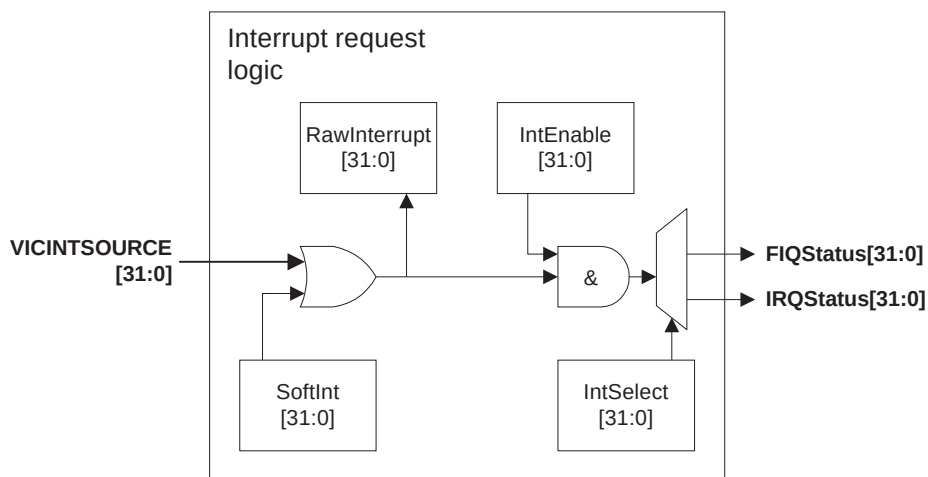


Figure 2-2 Interrupt request logic

2.1.2 Nonvectored FIQ interrupt logic

The nonvectored FIQ interrupt logic generates the FIQ interrupt signal by combining the FIQ interrupt requests in the interrupt controller and any requests from daisy-chained interrupt controllers. Figure 2-3 shows a block diagram of the nonvectored FIQ interrupt logic.

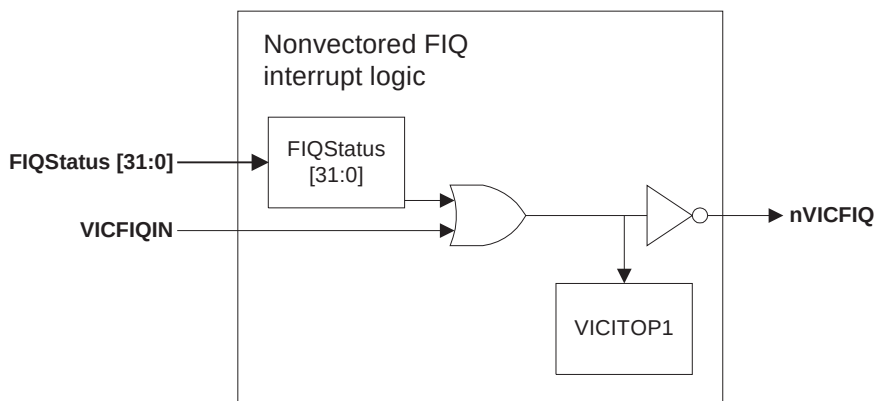


Figure 2-3 Nonvectored FIQ interrupt logic

2.1.3 Nonvectored IRQ interrupt logic

The nonvectored IRQ interrupt logic combines the nonvectored interrupt requests to generate the nonvectored IRQ interrupt signal. This signal is then sent to the IRQ vector address and priority logic. Figure 2-4 shows a block diagram of the nonvectored IRQ interrupt logic.

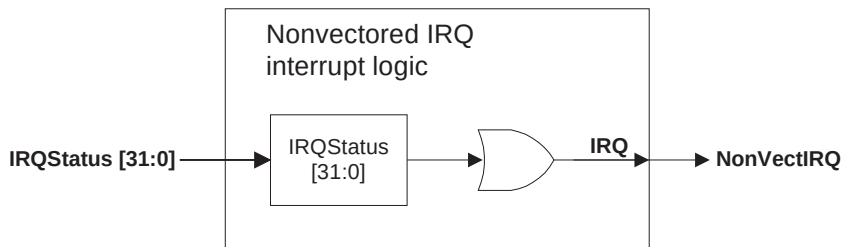


Figure 2-4 Nonvectored IRQ interrupt logic

2.1.4 Vectored interrupt block

There are 16 vectored interrupt blocks. The vectored interrupt blocks receive the IRQ interrupt requests and set **VectIRQx** if the following are true:

- the selected interrupt is active
- the selected interrupt is the currently highest requesting interrupt.

Each vectored interrupt block also provides a **VectorAddrx[31:0]** output for use in the interrupt priority block. Figure 2-5 shows a block diagram of two of the vectored interrupt blocks.

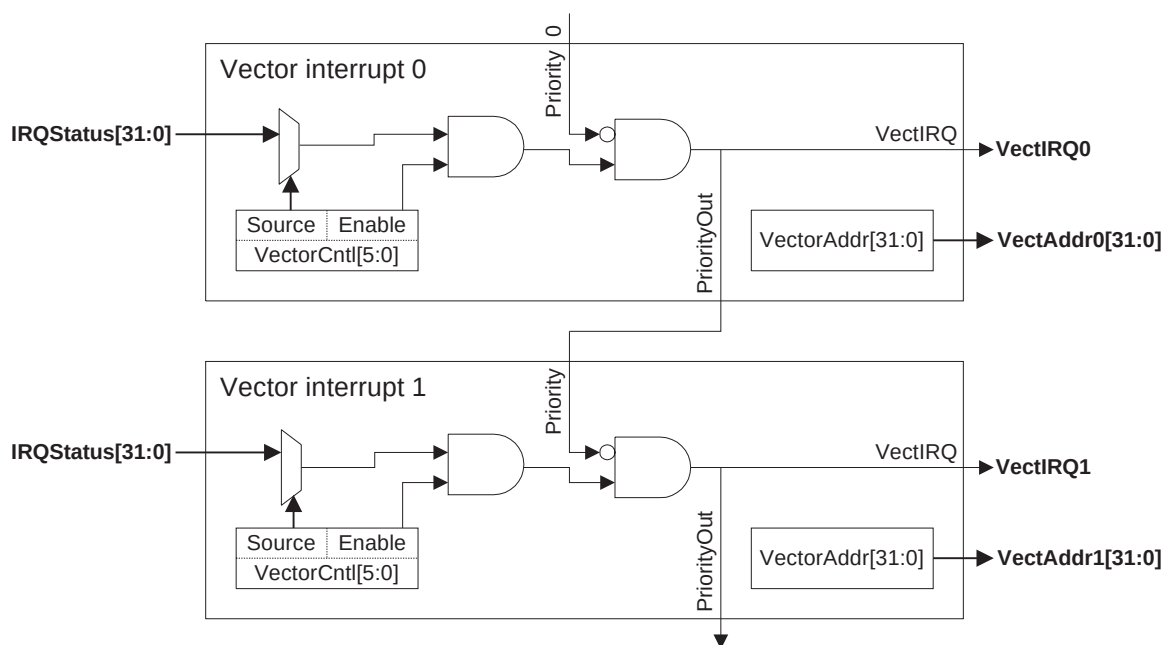


Figure 2-5 Vectored interrupt block

2.1.5 Interrupt priority logic

The interrupt priority block prioritizes the following requests:

- nonvectored interrupt requests
- vectored interrupt requests
- external interrupt requests.

The highest-priority request generates an IRQ interrupt if the interrupt is not currently being serviced. Figure 2-6 shows a block diagram of the interrupt priority logic.

———— **Note** ————

nVICIRQIN is the daisy-chained IRQ request input.

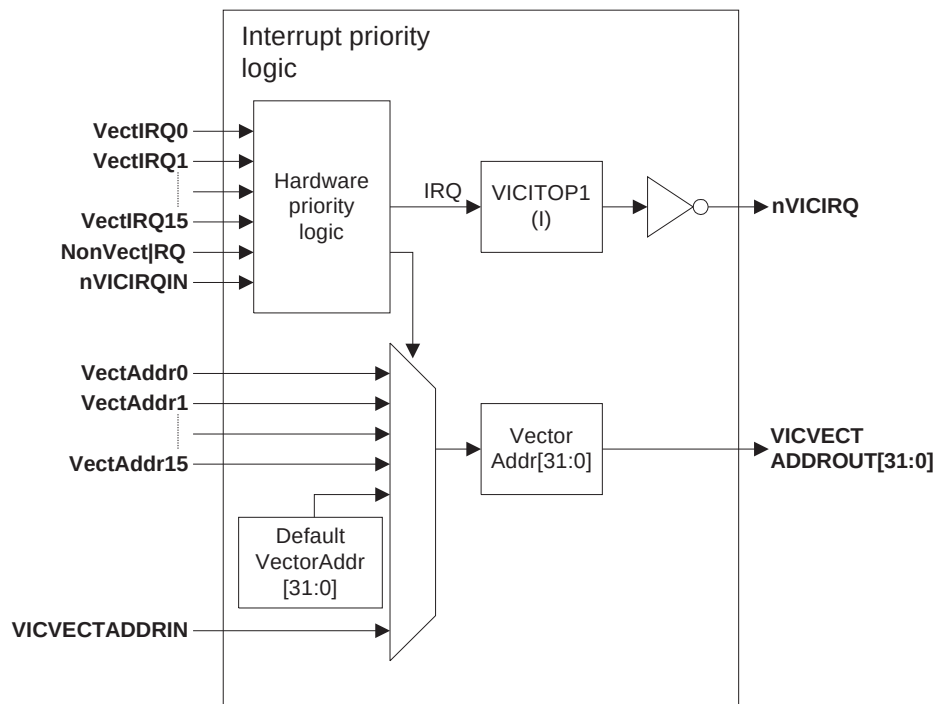


Figure 2-6 Interrupt priority logic

2.1.6 Vectored interrupts

A vectored interrupt is only generated if the following are true:

- it is enabled in the interrupt enable register, VICIntEnable
- it is set to generate an IRQ interrupt in the interrupt select register, VICIntSelect
- it is enabled in the relevant vector control register, VICVectCntl[0-15].

This prevents multiple interrupts being generated from a single interrupt request if the controller is incorrectly programmed.

2.1.7 Software interrupts

The software can control the source interrupt lines to generate software interrupts. These interrupts are generated before interrupt masking, in the same way as external source interrupts. Software interrupts are cleared by writing to the software interrupt clear register, VICSoftIntClear (see *Software interrupt clear register, VICSoftIntClear* on page 3-9). This is normally done at the end of the interrupt service routine.

2.2 PrimeCell VIC operation

The operation of the PrimeCell VIC is described in the following sections:

- *Vectored interrupt flow sequence*
- *Simple interrupt flow.*

2.2.1 Vectored interrupt flow sequence

The following procedure shows the sequence for the vectored interrupt flow:

1. An interrupt occurs.
2. The ARM processor branches to either the IRQ or FIQ interrupt vector.
3. If the interrupt is an IRQ, read the VICVectAddr register and branch to the interrupt service routine. This can be done using an LDR PC instruction. Reading the VICVectorAddr register updates the interrupt controllers hardware priority register.
4. Stack the workspace so that IRQ interrupts can be re-enabled.
5. Enable the IRQ interrupts so that a higher priority can be serviced.
6. Execute the *Interrupt Service Routine (ISR)*.
7. Clear the requesting interrupt in the peripheral, or write to the VICSoftIntClear register if the request was generated by a software interrupt.
8. Disable the interrupts and restore the workspace.
9. Write to the VICVectAddr register. This clears the respective interrupt in the internal interrupt priority hardware.
10. Return from the interrupt. This re-enables the interrupts.

2.2.2 Simple interrupt flow

The following procedure shows how you can use the interrupt controller without using vectored interrupts or the interrupt priority hardware. For example, you can use it for debugging.

1. An interrupt occurs.
2. Branch to IRQ or FIQ interrupt vector.
3. Branch to the interrupt handler.

4. Interrogate the VICIRQStatus register to determine which source generated the interrupt, and prioritize the interrupts if there are multiple active interrupt sources. This takes a number of instructions to compute.
5. Branch to the correct ISR.
6. Execute the ISR.
7. Clear the interrupt. If the request was generated by a software interrupt, the VICSoftIntClear register must be written to.
8. Check the VICIRQStatus register to ensure that no other interrupt is active. If there is an active request go to Step 4.
9. Return from the interrupt.

———— **Note** ————

If the above flow is used, you must not read or write to the VICVectorAddr register.

—————

2.3 PrimeCell VIC connectivity

The PrimeCell VIC is normally used as a standalone interrupt controller. Where required, you can daisy-chain a second PrimeCell VIC.

Note

The interrupt latency increases if daisy chaining is used (see *Daisy-chained interrupts* on page 3-21).

The Primecell VIC is connected to the processor as a standard AHB slave, with the FIQ and IRQ signals connected to the FIQ and IRQ inputs on the processor. The interrupt request lines from the peripheral are connected to the **VICINTSOURCE** inputs of the PrimeCell VIC. To ensure that the vector address register (see *Vector address register*, *VICVectAddr* on page 3-10) can be read in a single instruction, the PrimeCell VIC must be located in the upper 4K of memory, at 0xffff_f000.

Note

If the PrimeCell VIC is located at a different address, interrupt latency will be increased.

The connectivity for the two options is described in the following sections:

- *Standalone interrupt controller*
- *Daisy-chained interrupt controller* on page 2-13.

2.3.1 Standalone interrupt controller

If the PrimeCell VIC is used as a standalone interrupt controller, **nVICIRQIN** and **nVICFIQIN** must be tied HIGH, and **VICVECTADDRIN[31:0]** tied LOW. Figure 2-7 on page 2-13 shows the connections between the PrimeCell VIC and the processor when used as a standalone interrupt controller.

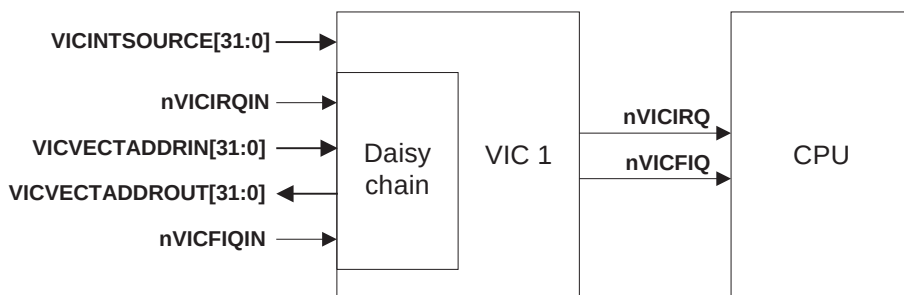


Figure 2-7 Standalone interrupt controller connectivity

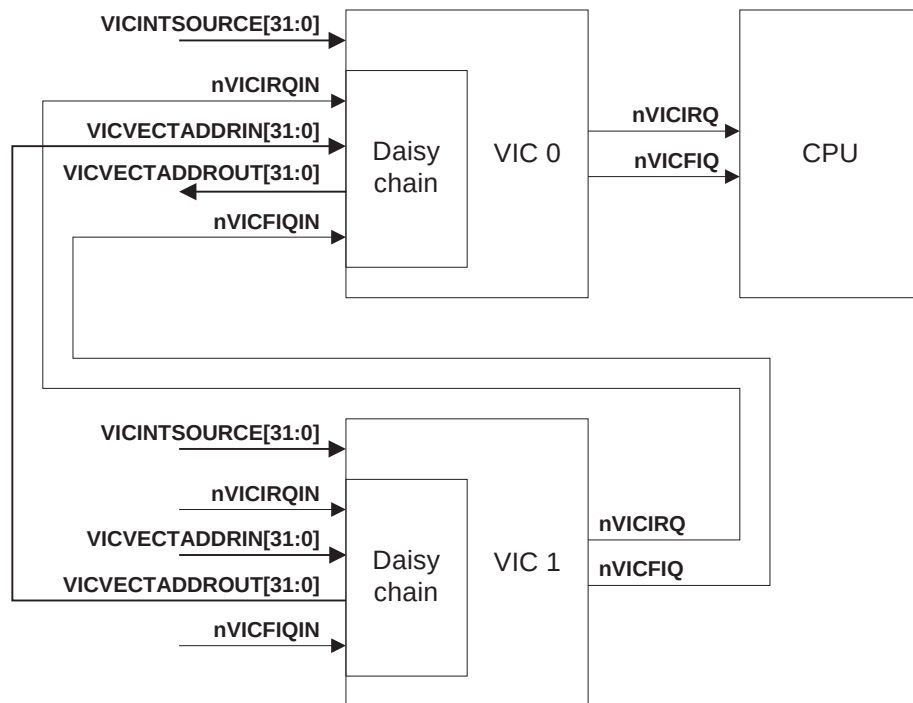
2.3.2 Daisy-chained interrupt controller

If the PrimeCell VIC is used in a daisy chain, connect the signals between the VICs as follows:

- **nVICIRQIN** on the first VIC to the **nVICIRQ** output of the second VIC
- **nVICFIQ** on the first VIC to the **nVICFIQ** output of the second VIC
- **VICVECTADDRIN[31:0]** on the first VIC to the **VICVECTADDROUT[31:0]** of the second VIC.

Connect the final VIC in the chain (the VIC furthest from the processor) as described in *Standalone interrupt controller* on page 2-12.

Figure 2-8 on page 2-14 shows the connections between the PrimeCell VICs and the processor when used in a daisy chain.

**Figure 2-8 Daisy-chained interrupt controller connectivity**

Chapter 3

Programmer's Model

This chapter describes the ARM PrimeCell VIC (PL190) registers, and provides details needed when programming the microcontroller. It contains the following sections:

- *About the programmer's model* on page 3-2
- *Summary of PrimeCell VIC registers* on page 3-3
- *Register descriptions* on page 3-6
- *Interrupt latency* on page 3-19
- *Interrupt priority* on page 3-22.

3.1 About the programmer's model

To ensure that the vector address register (see *Vector address register, VICVectAddr* on page 3-10) can be read in a single instruction, the PrimeCell VIC base address must be 0xFFFFF000, the upper 4K of memory. Placing the PrimeCell VIC anywhere else in memory increases interrupt latency as the ARM processor is unable to access the VICVectorAddr register using a single instruction.

The offset of any particular register from the base address is fixed.

3.2 Summary of PrimeCell VIC registers

The PrimeCell VIC registers are shown in Table 3-1.

Table 3-1 PrimeCell VIC register summary

Address	Type	Width	Reset value	Name	Description
VIC base +0x000	Read	32	0x00000000	VICIRQStatus	IRQ status register
VIC base +0x004	Read	32	0x00000000	VICFIQStatus	FIQ status register
VIC base +0x008	Read	32	-	VICRawIntr	Raw interrupt status register
VIC base +0x00C	Read/write	32	0x00000000	VICIntSelect	Interrupt select register
VIC base +0x010	Read/write	32	0x00000000	VICIntEnable	Interrupt enable register
VIC base +0x014	Write	32	-	VICIntEnClear	Interrupt enable clear register
VIC base +0x018	Read/write	32	0x00000000	VICSoftInt	Software interrupt register
VIC base +0x01C	Write	32	-	VICSoftIntClear	Software interrupt clear register
VIC base +0x020	Read/write	1	0x0	VICProtection	Protection enable register
VIC base +0x030	Read/write	32	0x00000000	VICVectAddr	Vector address register
VIC base +0x034	Read/write	32	0x00000000	VICDefVectAddr	Default vector address register
VIC base +0x100	Read/write	32	0x00000000	VICVectAddr0	Vector address 0 register
VIC base +0x104	Read/write	32	0x00000000	VICVectAddr1	Vector address 1 register
VIC base +0x108	Read/write	32	0x00000000	VICVectAddr2	Vector address 2 register
VIC base +0x10C	Read/write	32	0x00000000	VICVectAddr3	Vector address 3 register
VIC base +0x110	Read/write	32	0x00000000	VICVectAddr4	Vector address 4 register
VIC base +0x114	Read/write	32	0x00000000	VICVectAddr5	Vector address 5 register
VIC base +0x118	Read/write	32	0x00000000	VICVectAddr6	Vector address 6 register
VIC base +0x11C	Read/write	32	0x00000000	VICVectAddr7	Vector address 7 register
VIC base +0x120	Read/write	32	0x00000000	VICVectAddr8	Vector address 8 register
VIC base +0x124	Read/write	32	0x00000000	VICVectAddr9	Vector address 9 register
VIC base +0x128	Read/write	32	0x00000000	VICVectAddr10	Vector address 10 register
VIC base +0x12C	Read/write	32	0x00000000	VICVectAddr11	Vector address 11 register

Table 3-1 PrimeCell VIC register summary (continued)

Address	Type	Width	Reset value	Name	Description
VIC base +0x130	Read/write	32	0x00000000	VICVectAddr12	Vector address 12 register
VIC base +0x134	Read/write	32	0x00000000	VICVectAddr13	Vector address 13 register
VIC base +0x138	Read/write	32	0x00000000	VICVectAddr14	Vector address 14 register
VIC base +0x13C	Read/write	32	0x00000000	VICVectAddr15	Vector address 15 register
VIC base +0x200	Read/write	6	0x00	VICVectCntl0	Vector control 0 register
VIC base +0x204	Read/write	6	0x00	VICVectCntl1	Vector control 1 register
VIC base +0x208	Read/write	6	0x00	VICVectCntl2	Vector control 2 register
VIC base +0x20C	Read/write	6	0x00	VICVectCntl3	Vector control 3 register
VIC base +0x210	Read/write	6	0x00	VICVectCntl4	Vector control 4 register
VIC base +0x214	Read/write	6	0x00	VICVectCntl5	Vector control 5 register
VIC base +0x218	Read/write	6	0x00	VICVectCntl6	Vector control 6 register
VIC base +0x21C	Read/write	6	0x00	VICVectCntl7	Vector control 7 register
VIC base +0x220	Read/write	6	0x00	VICVectCntl8	Vector control 8 register
VIC base +0x224	Read/write	6	0x00	VICVectCntl9	Vector control 9 register
VIC base +0x228	Read/write	6	0x00	VICVectCntl10	Vector control 10 register
VIC base +0x22C	Read/write	6	0x00	VICVectCntl11	Vector control 11 register
VIC base +0x230	Read/write	6	0x00	VICVectCntl12	Vector control 12 register
VIC base +0x234	Read/write	6	0x00	VICVectCntl13	Vector control 13 register
VIC base +0x238	Read/write	6	0x00	VICVectCntl14	Vector control 14 register
VIC base +0x23C	Read/write	6	0x00	VICVectCntl15	Vector control 15 register
VIC base +0x300	Read/write	1	-	VICITCR	Test control register
VIC base +0x304	Read	2	-	VICITIP1	Test input register (nVICIRQIN/nVICFIQIN)
VIC base +0x308	Read	32	-	VICITIP2	Test input register (VICVECTADDRIN)

Table 3-1 PrimeCell VIC register summary (continued)

Address	Type	Width	Reset value	Name	Description
VIC base +0x30C	Read	2	0x0	VICITOP1	Test output register (nVICIRQ/nVICFIQ)
VIC base +0x310	Read	32	0x00000000	VICITOP2	Test output register (VICVECTADDRROUT)
VIC base +0xFE0	Read	8	0x80	VICPeriphID0	Peripheral identification register bits 7:0
VIC base +0xFE4	Read	8	0x11	VICPeriphID1	Peripheral identification register bits 15:8
VIC base +0xFE8	Read	8	0x04	VICPeriphID2	Peripheral identification register bits 23:16
VIC base +0xFEC	Read	8	0x00	VICPeriphID3	Peripheral identification register bits 31:24
VIC base +0xFF0	Read	8	0x0D	VICPCellID0	PrimeCell identification register bits 7:0
VIC base +0xFF4	Read	8	0xF0	VICPCellID1	PrimeCell identification register bits 15:8
VIC base +0xFF8	Read	8	0x05	VICPCellID2	PrimeCell identification register bits 23:16
VIC base +0xFFC	Read	8	0xB1	VICPCellID3	PrimeCell identification register bits 31:24

3.3 Register descriptions

The following registers are described in this section:

- *IRQ status register, VICIRQStatus*
- *FIQ status register, VICFIQStatus* on page 3-7
- *Raw interrupt status register, VICRawIntr* on page 3-7
- *Interrupt select register, VICIntSelect* on page 3-7
- *Interrupt enable register, VICIntEnable* on page 3-8
- *Interrupt enable clear register, VICIntEnClear* on page 3-8
- *Software interrupt register, VICSoftInt* on page 3-8
- *Software interrupt clear register, VICSoftIntClear* on page 3-9
- *Protection enable register, VICProtection* on page 3-9
- *Vector address register, VICVectAddr* on page 3-10
- *Default vector address register, VICDefVectAddr* on page 3-10
- *Vector address registers, VICVectAddr[0-15]* on page 3-11
- *Vector control registers, VICVectCntl[0-15]* on page 3-11
- *Test output register (nVICIRQ/nVICFIQ), VICITOP1* on page 3-13
- *Test output register (VICVECTADDRROUT), VICITOP2* on page 3-13
- *Test input register (nVICIRQIN/nVICFIQIN), VICITIP1* on page 3-12
- *Test input register (VICVECTADDRIN), VICITIP2* on page 3-12
- *Test control register, VICITCR* on page 3-12
- *Peripheral identification registers, VICPeriphID0-3* on page 3-14
- *PrimeCell identification registers, VICPCellID0-3* on page 3-17.

3.3.1 IRQ status register, VICIRQStatus

The VICIRQStatus register provides the status of interrupts [31:0] after IRQ masking. Table 3-2 shows the bit assignment of the VICIRQStatus register.

Table 3-2 VICIRQStatus register

Bits	Name	Type	Function
31:0	IRQStatus	Read	Shows the status of the interrupts after masking by the VICIntEnable and VICIntSelect registers. A HIGH bit indicates that the interrupt is active, and generates an interrupt to the processor.

3.3.2 FIQ status register, VICFIQStatus

The VICFIQStatus register provides the status of the interrupts after FIQ masking. Table 3-3 shows the bit assignment of the VICFIQStatus register.

Table 3-3 VICFIQStatus register

Bits	Name	Type	Function
31:0	FIQStatus	Read	Shows the status of the interrupts after masking by the VICIntEnable and VICIntSelect registers. A HIGH bit indicates that the interrupt is active, and generates an interrupt to the processor.

3.3.3 Raw interrupt status register, VICRawIntr

The VICRawIntr register provides the status of the source interrupts (and software interrupts) to the interrupt controller. Table 3-4 shows the bit assignment of the VICRawIntr register.

Table 3-4 VICRawIntr register

Bits	Name	Type	Function
31:0	RawInterrupt	Read	Shows the status of the interrupts before masking by the enable registers. A HIGH bit indicates that the appropriate interrupt request is active before masking.

3.3.4 Interrupt select register, VICIntSelect

The VICIntSelect register selects whether the corresponding interrupt source generates an FIQ or an IRQ interrupt. Table 3-5 shows the bit assignment of the VICIntSelect register.

Table 3-5 VICIntSelect register

Bits	Name	Type	Function
31:0	IntSelect	Read/write	Selects type of interrupt for interrupt request: 1 = FIQ interrupt 0 = IRQ interrupt.

3.3.5 Interrupt enable register, VICIntEnable

The VICIntEnable register enables the interrupt request lines, by masking the interrupt sources for the IRQ interrupt. Table 3-6 shows the bit assignment of the VICIntEnable register.

Table 3-6 VICIntEnable register

Bits	Name	Type	Function
31:0	IntEnable	Read/write	Enables the interrupt request lines: 1 = Interrupt enabled. Allows interrupt request to processor. 0 = Interrupt disabled. On reset, all interrupts are disabled. A HIGH bit sets the corresponding bit in the VICIntEnable register. A LOW bit has no effect.

3.3.6 Interrupt enable clear register, VICIntEnClear

The VICIntEnClear register clears bits in the VICIntEnable register. Table 3-7 shows the bit assignment of the VICIntEnClear register.

Table 3-7 VICIntEnClear register

Bits	Name	Type	Function
31:0	IntEnable Clear	Write	Clears bits in the VICIntEnable register. A HIGH bit clears the corresponding bit in the VICIntEnable register. A LOW bit has no effect.

3.3.7 Software interrupt register, VICSoftInt

The VICSoftInt register is used to generate software interrupts. Table 3-8 shows the bit assignment of the VICSoftInt register.

Table 3-8 VICSoftInt register

Bits	Name	Type	Function
31:0	SoftInt	Read/write	Setting a bit generates a software interrupt for the specific source interrupt before interrupt masking. A HIGH bit sets the corresponding bit in the VICSoftInt register. A LOW bit has no effect.

3.3.8 Software interrupt clear register, VICSoftIntClear

The VICSoftIntClear register clears bits in the VICSoftInt register. Table 3-9 shows the bit assignment of the VICSoftIntClear register.

Table 3-9 VICSoftIntClear register

Bits	Name	Type	Function
31:0	SoftIntClear	Write	Clears bits in the VICSoftInt register. A HIGH bit clears the corresponding bit in the VICSoftInt register. A LOW bit has no effect.

3.3.9 Protection enable register, VICProtection

The VICProtection register enables or disables protected register access. Table 3-10 shows the bit assignment of the VICProtection register.

Table 3-10 VICProtection register

Bits	Name	Type	Function
31:1	Reserved	-	-
0	Protection	Read/write	Enables or disables protected register access. When enabled, only privileged mode accesses (reads and writes) can access the interrupt controller registers. When disabled, both User mode and privileged mode can access the registers. This register is cleared on reset, and can only be accessed in privileged mode.

———— **Note** ————

If the bus master cannot generate accurate protection information, leave this register in its reset state to allow User mode access.

3.3.10 Vector address register, VICVectAddr

The VICVectAddr register contains the *Interrupt Service Routine* (ISR) address of the currently active interrupt. Table 3-11 shows the bit assignment of the VICVectAddr register.

Table 3-11 VICVectAddr register

Bits	Name	Type	Function
31:0	VectorAddr	Read/write	Contains the address of the currently active ISR. Any writes to this register clear the interrupt.

———— **Note** —————

Reading from this register provides the address of the ISR, and indicates to the priority hardware that the interrupt is being serviced. Writing to this register indicates to the priority hardware that the interrupt has been serviced. The register should be used as follows:

- The ISR reads the VICVectAddr register when an IRQ interrupt is generated
- At the end of the ISR, the VICVectAddr register is written to, to update the priority hardware.

Reading or writing to the register at other times can cause incorrect operation.

3.3.11 Default vector address register, VICDefVectAddr

The VICDefVectAddr register contains the default ISR address. Table 3-12 shows the bit assignment of the VICDefVectAddr register.

Table 3-12 VICDefVectAddr register

Bits	Name	Type	Function
31:0	Default VectorAddr	Read/write	Contains the address of the default ISR handler.

3.3.12 Vector address registers, VICVectAddr[0-15]

The sixteen VICVectAddr[0-15] registers contain the ISR vector addresses. Table 3-13 shows the bit assignment of the VICVectAddr[0-15] registers.

Table 3-13 VICVectAddr[0-15] registers

Bits	Name	Type	Function
31:0	VectorAddr 0-15	Read/write	Contains ISR vector addresses.

3.3.13 Vector control registers, VICVectCntl[0-15]

The sixteen VICVectCntl[0-15] registers select the interrupt source for the vectored interrupt. Table 3-14 shows the bit assignment of the VICVectCntl[0-15] registers.

Table 3-14 VICVectCntl[0-15] registers

Bits	Name	Type	Function
31:6	Reserved	-	-
5	E	Read/write	Enables vector interrupt. This bit is cleared on reset.
4:0	IntSource	Read/write	Selects interrupt source. You can select any of the 32 interrupt sources.

Note

Vectored interrupts are only generated if the interrupt is enabled. The specific interrupt is enabled in the VICIntEnable register, and the interrupt is set to generate an IRQ interrupt in the VICIntSelect register. This prevents multiple interrupts being generated from a single request if the controller is incorrectly programmed.

3.3.14 Test control register, VICITCR

The VICITCR register selects test mode, and is cleared on reset. Table 3-15 shows the bit assignment of the VICITCR register.

Table 3-15 VICITCR register

Bits	Name	Type	Function
31:1	Reserved	-	-
0	ITEN	Read/write	Selects test mode, to use VICITIP test registers in place of input signals.

3.3.15 Test input register (nVICIRQIN/nVICFIQIN), VICITIP1

The VICITIP1 register indicates the status of the **nVICIRQIN** and **nVICFIQIN** daisy chain input lines. Table 3-16 shows the bit assignment of the VICITIP1 register.

Table 3-16 VICITIP1 register

Bits	Name	Type	Function
31:8	Reserved	-	-
7	I	Read	Indicates status of nVICIRQIN when VICITCR register is LOW
6	F	Read	Indicates status of nVICFIQIN when VICITCR register is LOW
5:0	Reserved	-	-

3.3.16 Test input register (VICVECTADDRIN), VICITIP2

The VICITIP2 register indicates the status of the **VICVECTADDRIN** daisy chain input lines. Table 3-17 shows the bit assignment of the VICITIP2 register.

Table 3-17 VICITIP2 register

Bits	Name	Type	Function
31:0	VectorAddrIn	Read	Indicates status of VICVECTADDRIN when VICITCR register is LOW.

3.3.17 Test output register (nVICIRQ/nVICFIQ), VICITOP1

The VICITOP1 register indicates the status of the **nVICIRQ** and **nVICFIQ** interrupt request lines to the processor. Table 3-18 shows the bit assignment of the VICITOP1 register.

Table 3-18 VICITOP1 register

Bits	Name	Type	Function
31:8	Reserved	-	-
7	I	Read	Status of nVICIRQ interrupt line. If set HIGH, the interrupt request is active.
6	F	Read	Status of nVICFIQ interrupt line. If set HIGH, the interrupt request is active.
5:0	Reserved	-	-

3.3.18 Test output register (VICVECTADDRROUT), VICITOP2

The VICITOP2 register indicates the status of the **VICVECTADDRROUT** lines from the interrupt controller. Table 3-19 shows the bit assignment of the VICITOP2 register.

Table 3-19 VICITOP2 register

Bits	Name	Type	Function
31:0	VectorAddr Out	Read	Indicates status of VICVECTADDRROUT from interrupt controller.

3.3.19 Peripheral identification registers, VICPeriphID0-3

The VICPeriphID0-3 registers are four 8-bit registers, that span address locations 0xFE0-0xFEC. The registers can conceptually be treated as a single 32-bit register. The read-only registers provide the following options of the peripheral:

Part number [11:0]

This identifies the peripheral. The three digit product code 0x90 is used for the PrimeCell VIC.

Designer [19:12]

This is the identification of the designer. ARM Ltd is 0x41 (ASCII A).

Revision number [23:20]

This is the revision number of the peripheral. The revision number starts from 0.

Configuration [31:24]

This is the configuration option of the peripheral. The configuration value is 0.

Figure 3-1 shows the bit assignment for the VICPeriphID0-3 registers.

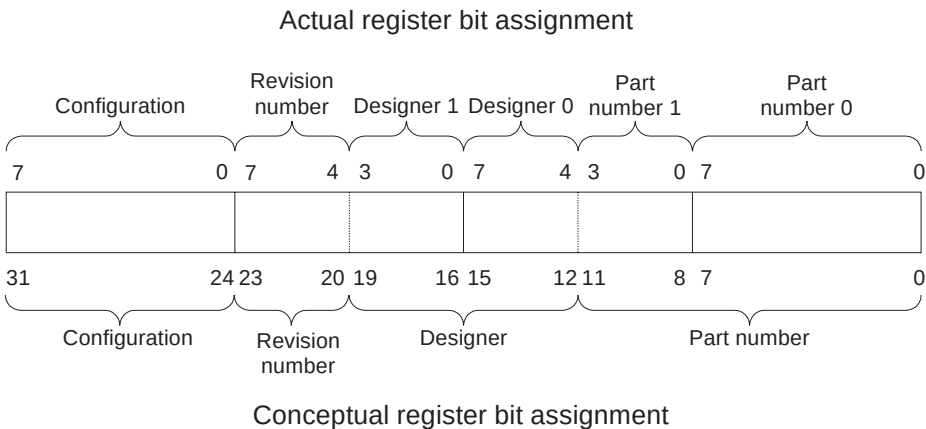


Figure 3-1 Peripheral identification register bit allocation

The four 8-bit peripheral identification registers are described in the following sections:

- *VICPeriphID0 register*
- *VICPeriphID1 register*
- *VICPeriphID2 register* on page 3-16
- *VICPeriphID3 register* on page 3-16.

VICPeriphID0 register

The VICPeriphID0 register is hard-coded and the fields within the register determine the reset value. Table 3-20 shows the bit assignment of the VICPeriphID0 register.

Table 3-20 VICPeriphID0 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros.
7:0	Partnumber0	Read	These bits read back as 0x90.

VICPeriphID1 register

The VICPeriphID1 register is hard-coded and the fields within the register determine the reset value. Table 3-21 shows the bit assignment of the VICPeriphID1 register.

Table 3-21 VICPeriphD1 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros.
7:4	Designer0	Read	These bits read back as 0x1.
3:0	Partnumber1	Read	These bits read back as 0x1.

VICPeriphID2 register

The VICPeriphID2 register is hard-coded and the fields within the register determine the reset value. Table 3-22 shows the bit assignment of the VICPeriphID2 register.

Table 3-22 VICPeriphID2 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros.
7:4	Revision	Read	These bits read back as 0x0.
3:0	Designer1	Read	These bits read back as 0x4.

VICPeriphID3 register

The VICPeriphID3 register is hard-coded and the fields within the register determine the reset value. Table 3-23 shows the bit assignment of the VICPeriphID3 register.

Table 3-23 VICPeriphID3 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros.
7:0	Configuration	Read	These bits read back as 0x0.

3.3.20 PrimeCell identification registers, VICPCellIID0-3

The VICPCellIID0-3 registers are four 8-bit registers, that span address locations 0xFF0-0xFFC. The read-only register can conceptually be treated as a single 32-bit register. The register is used as a standard cross-peripheral identification system. Figure 3-2 shows the bit assignment for the VICPCellIID0-3 registers.

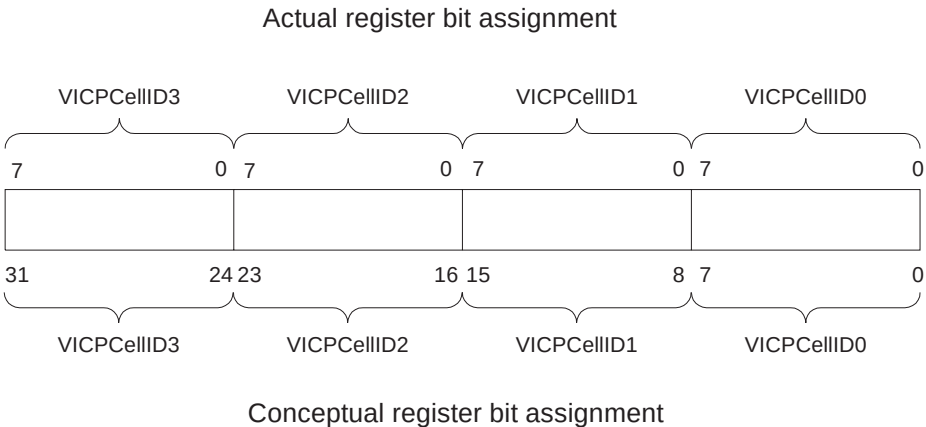


Figure 3-2 PrimeCell identification register bit assignment

The four 8-bit registers are described in the following subsections:

- *VICPCellIID0 register*
- *VICPCellIID1 register* on page 3-18
- *VICPCellIID2 register* on page 3-18
- *VICPCellIID3 register* on page 3-18.

VICPCellIID0 register

The VICPCellIID0 register is hard-coded and the fields within the register determine the reset value. Table 3-24 shows the bit assignment of the VICPCellIID0 register.

Table 3-24 VICPCellIID0 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros.
7:0	VICPCellIID0	Read	These bits read back as 0x0D.

VICPCellID1 register

The VICPCellID1 register is hard-coded and the fields within the register determine the reset value. Table 3-25 shows the bit assignment of the VICPCellID1 register.

Table 3-25 VICPCellID1 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros.
7:0	VICPCellID1	Read	These bits read back as 0xF0.

VICPCellID2 register

The VICPCellID2 register is hard-coded and the fields within the register determine the reset value. Table 3-26 shows the bit assignment of the VICPCellID2 register.

Table 3-26 VICPCellID2 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros.
7:0	VICPCellID2	Read	These bits read back as 0x05.

VICPCellID3 register

The VICPCellID3 register is hard-coded and the fields within the register determine the reset value. Table 3-27 shows the bit assignment of the VICPCellID3 register.

Table 3-27 VICPCellID3 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros.
7:0	VICPCellID3	Read	These bits read back as 0xB1.

3.4 Interrupt latency

The calculations in this section show the number of cycles required to service interrupts, using the following types of interrupt:

- *FIQ interrupts*
- *IRQ interrupts* on page 3-20
- *Fast IRQ interrupts* on page 3-20
- *Daisy-chained interrupts* on page 3-21.

———— **Note** ————

The calculations are based on the assumption that the ISRs are in 0 wait state memory.

————

3.4.1 FIQ interrupts

FIQ interrupts have the highest priority in the PrimeCell VIC, and are not nested. In FIQ mode, seven 32-bit registers are banked into the system. This allows the PrimeCell VIC to process the interrupt as quickly as possible. Table 3-28 shows the worst case cycles for FIQ interrupts.

Table 3-28 FIQ interrupt latency

Event	Worst case
Interrupt synchronization	3 cycles
Worst case instruction execution This assumes that a standard switch is used to reduce STM and LDM. This can be further reduced to 7 cycles, and data aborts are avoided.	7 cycles
Entry to first instruction	2 cycles
Total	12 cycles

———— **Note** ————

For best results, start the FIQ handler at the FIQ vector address, 0x1c.

————

3.4.2 IRQ interrupts

In IRQ mode, interrupt levels can be nested lower than the highest priority FIQ interrupt level. To provide this nesting, the return address, stored in the *Link Register* (LR), and the status register, stored in the *Saved Processor Status Register* (SPSR) must be available before further IRQ interrupts can be accepted. This increases the interrupt latency, but provides a scalable nested interrupt system. Table 3-29 shows the worst case cycles for IRQ interrupts.

Table 3-29 IRQ interrupt latency

Event	Worst case
Interrupt synchronization	3 cycles
Worst case interrupt disable period	10 cycles
Entry to first instruction	2 cycles
Nesting (assuming single-state AHB)	10 cycles
Total	25 cycles

3.4.3 Fast IRQ interrupts

Fast IRQ mode is similar to IRQ mode, except that the highest-level IRQ interrupt handler assumes that no other IRQ interrupt occurs during its operation, and therefore the LR and SPSR are not required. Table 3-30 shows the worst case cycles for fast IRQ interrupts.

Table 3-30 Fast IRQ interrupt latency

Event	Worst case
Interrupt synchronization	3 cycles
Worst case interrupt disable period	10 cycles
Entry to first instruction	2 cycles
Load IRQ vector into PC	5 cycles
Total	20 cycles

3.4.4 Daisy-chained interrupts

Because of the additional read required to read both the primary VICVectAddr register and the daisy-chained VICVectAddr register, the worst-case latency of the primary PrimeCell VIC is increased by one cycle, to 26 cycles. The worst-case latency for the secondary, daisy-chained PrimeCell VIC is increased by two cycles, to 27 cycles. This latency is applicable to any number of secondary PrimeCell VICs. See *Daisy-chained vectored interrupt service routine* on page B-6 for more information.

3.5 Interrupt priority

The interrupt priority is regulated by the hardware. FIQ interrupts have the highest priority, followed by vectored interrupt 0 to vectored interrupt 15. Nonvectored interrupts have the lowest priority.

To reduce interrupt latency (see *Interrupt latency* on page 3-19), you can re-enable the IRQ interrupts in the processor after the *Interrupt Service Routine* (ISR) is entered. In this case, the current ISR is interrupted and the higher-priority ISR will be executed. The PrimeCell VIC then only allows a higher priority interrupt than the interrupt currently being serviced. If a higher priority interrupt goes active the current ISR is interrupted and the higher-priority ISR is executed.

Before the interrupt enable bits in the processor can be reenabled, the LR and SPSR must be saved, preferably on a software stack. When the ISR is exited, the interrupts must be disabled, the LR and SPSR reloaded, and the vector address register, VICVectAddr, written to (see *Vectored interrupt service routine* on page B-6).

When PrimeCell VICs are daisy-chained the interrupt priority is as follows:

- the FIQ interrupts
- the primary VIC vectored interrupts
- the primary VIC nonvectored interrupts
- the daisy chained VIC vectored interrupts
- the daisy chained VIC nonvectored interrupts.

Chapter 4

Programmer's Model for Test

This chapter describes the additional logic for functional verification and provisions made for production testing. It contains the following sections:

- *PrimeCell VIC test harness overview* on page 4-2
- *Scan testing* on page 4-3
- *Test registers* on page 4-4
- *Integration tests* on page 4-7
- *Integration test summary* on page 4-11.

4.1 PrimeCell VIC test harness overview

The additional logic for functional verification and production testing allows:

- capture of input signals to the block
- stimulation of the output signals.

The integration vectors provide a way of verifying that the PrimeCell VIC is correctly wired into a system. This is done by separately testing two groups of signals:

AMBA signals

These are tested by checking the connections of all the address and data bits.

Intra-chip signals

The tests for these signals are system-specific, and enable you to write the necessary tests. Additional logic is implemented allowing you to read and write to each intra-chip input/output signal.

These test features are controlled by test registers. This allows you to test the PrimeCell VIC in isolation from the rest of the system using only transfers from the AMBA AHB.

Off-chip test vectors are supplied using a 32-bit parallel *External Bus Interface* (EBI) and converted to internal AMBA bus transfers. The application of test vectors is controlled through the *Test Interface Controller* (TIC) AMBA bus master module.

4.2 Scan testing

The PrimeCell VIC is designed to simplify:

- insertion of scan test cells
- use of *Automatic Test Pattern Generation* (ATPG).

This provides an alternative method of manufacturing test.

4.3 Test registers

The PrimeCell VIC test registers are memory-mapped as shown in Table 4-1.

Table 4-1 Test registers memory map

Address	Type	Width	Reset value	Name	Description
VIC base + 0x300	Read/write	1	-	VICITCR	Test control register
VIC base + 0x304	Read	2	0x0	VICITIP1	Test input register (nVICIRQIN/nVICFIQIN)
VIC base + 0x308	Read	32	-	VICITIP2	Test input register (VICVECTADDRIN)
VIC base + 0x30C	Read	2	0x0	VICITOP1	Test output register (nVICIRQ/nVICFIQ)
VIC base + 0x310	Read	32	0x00000000	VICITOP2	Test output register (VICVECTADDRROUT)

The following registers are described in this section:

- *Test control register, VICITCR*
- *Integration test input register (nVICIRQIN/nVICFIQIN), VICITIP1* on page 4-5
- *Integration test input register (VICVECTADDRIN), VICITIP2* on page 4-5
- *Integration test output register (nVICIRQ/nVICFIQ), VICITOP1* on page 4-6
- *Integration test output register (VICVECTADDRROUT), VICITOP2* on page 4-6.

4.3.1 Test control register, VICITCR

VICITCR is a single-bit test control register. The ITEN bit in this register controls the input test multiplexors. Table 4-2 shows the bit assignment of the VICITCR.

Table 4-2 VICITCR register

Bits	Name	Type	Description
31:1	Reserved	-	-
0	ITEN	Read/write	Integration test enable: 0 = normal mode 1 = test mode.

4.3.2 Integration test input register (nVICIRQIN/nVICFIQIN), VICITIP1

VICITIP1 is a 2-bit register that returns the values of the **nVICIRQIN** and **nVICFIQIN** inputs. Table 4-3 shows the bit assignment of the VICITIP1 register.

Table 4-3 VICITIP1 register

Bits	Name	Type	Description
31:8	Reserved	-	-
7	nVICIRQIN	Read	Reads return the value on nVICIRQIN when the VICITCR register is LOW.
6	nVICFIQIN	Read	Reads return the value on nVICFIQIN when the VICITCR register is LOW.
5:0	Reserved	-	-

4.3.3 Integration test input register (VICVECTADDRIN), VICITIP2

VICITIP2 is a 32-bit register that returns the value of the **VICVECTADDRIN** input. Table 4-4 shows the bit assignment of the VICITIP2 register.

Table 4-4 VICITIP2 register

Bits	Name	Type	Description
31:0	VICVECTADDRIN	Read	Reads return the value on VICVECTADDRIN when the VICITCR register is LOW.

4.3.4 Integration test output register (nVICIRQ/nVICFIQ), VICITOP1

VICITOP1 is a 2-bit register that controls the **nVICIRQ** and **nVICFIQ** outputs. Table 4-5 shows the bit assignment of the VICITOP1 register.

Table 4-5 VICITOP1 register

Bits	Name	Type	Description
31:8	Reserved	-	-
7	VICIRQ	Read	Reads return the value on the internal VICIRQ line. This is the pre-inverted version of the final output, and is inverted to create the final nVICIRQ output.
6	VICFIQ	Read	Reads return the value on the internal VICFIQ line. This is the pre-inverted version of the final output, and is inverted to create the final nVICFIQ output.
5:0	Reserved	-	-

4.3.5 Integration test output register (VICVECTADDRROUT), VICITOP2

VICITOP2 is a 32-bit register that controls the **VICVECTADDRROUT** output. Table 4-6 shows the bit assignment of the VICITOP2 register.

Table 4-6 VICITOP2 register

Bits	Name	Type	Description
31:0	VICVECTADDRROUT	Read	Reads return the value on the VICVECTADDRROUT lines.

4.4 Integration tests

The integration tests verify that the VIC is wired into the system correctly. The following tests are described:

- *Integration testing of AHB pins*
- *Integration testing of intra-chip inputs* on page 4-8
- *Integration testing of intra-chip outputs* on page 4-10.

4.4.1 Integration testing of AHB pins

Use this test to verify the following AHB signals. This is done using a sequence of register accesses.

HCLK and HRESETn

Verify that these signals toggle by ensuring that the subsequent tests run correctly.

————— **Note** —————

This test starts with an assertion and then a deassertion of **HRESETn**.

-
- | | |
|------------------|---|
| HADDR | Deassert HRESETn , and perform a read access on all the internal registers. Verify that 1s and 0s appear on HADDR . |
| HTRANS[1] | Initiate an NSEQ (HTRANS[1] = 1) and an IDLE (HTRANS[1] = 0) transfer, and verify that 1 and 0 appear on HTRANS[1] . |
| HWRITE | Toggle HWRITE by performing read and write accesses to the VIC. |
| HSIZE | The VIC only supports word accesses (HSIZE = 2). Perform word and halfword accesses to the internal registers, and verify that 1s and 0s appear on the HSIZE lines. |
| HPROT | Perform User and Supervisor mode accesses, and verify that 1 and 0 appear on HPROT . Toggle the signal as follows: <ul style="list-style-type: none"> • Configure the VIC in Protected mode. • Access the VIC in Supervisor mode by writing a data value of 0x55555555 to the VICDefVectAddr register. • Access the VIC in User mode and modify the register. • Check the register content and verify that it has not been modified. |

HWDATA and HRDATA

Perform read and write accesses with different data patterns (0x55555555 and 0xAAAAAAAA). Verify the connectivity by validating these accesses.

HSELVIC Perform a write to a location, with **HADDR[11:2]** the same as the offset address of the VICIntSelect register, but with a different base address to the VIC. Verify that 1 and 0 appear on the signal. Verify the connectivity of **HSELVIC** by checking for unmodified data in the VICIntSelect register.

4.4.2 Integration testing of intra-chip inputs

Use this test for the following inputs:

- **nVICFIQIN**
- **nVICIRQIN**
- **VICVECTADDRIN**
- **VICINTSOURCE**.

When you run integration tests with the PrimeCell VIC as part of an integrated system:

1. Write a 0 to the ITEN bit in the VICITCR register. This selects the normal path from the intra-chip inputs to the internal VIC signals.
2. Verify that 1 and 0 appear on the inputs by programming their sources as necessary.
3. Where multiple VICs are daisy-chained in a system, verify the **nVICFIQIN**, **nVICIRQIN**, and **VICVECTADDRIN** inputs by writing to the internal registers of the first VIC, and reading back from the VICITIP1 and VICITIP2 registers of the next VIC. If the source of these signals is not another VIC, program the relevant controller as necessary.
4. The **VICINTSOURCE** input is typically sourced from other interrupt-capable peripherals in the system. To verify these inputs, program the peripherals as necessary, and read back the expected values from the VICRawIntr register.

Figure 4-1 on page 4-9 shows details of the implementation of the input integration test harness.

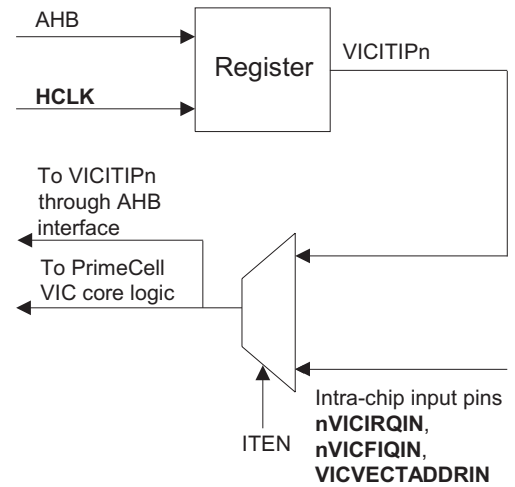


Figure 4-1 Input integration test harness

The ITEN bit is used as the control bit for the multiplexor, which is used in the read path of the intra-chip inputs:

- if the ITEN bit is deasserted, the intra-chip inputs are routed.
- if the ITEN bit is not deasserted, the stored register value is driven on the internal line.

4.4.3 Integration testing of intra-chip outputs

Use this test for the following outputs:

- **nVICFIQ**
- **nVICIRQ**
- **VICVECTADDRROUT**.

When you run integration tests with the PrimeCell VIC in a standalone test setup:

1. Program the PrimeCell VIC to set the intra-chip outputs to both 0 and 1.
2. Read the expected values back from the VICITOP1 and VICITOP2 registers. For the **nVICIRQ** and **nVICFIQ** outputs, a 1 on the output pin will be shown as a 0 in the VICITOP1 register, and a 0 as a 1.

When you run integration tests with the PrimeCell VIC as part of an integrated system:

1. Program the PrimeCell VIC to set the intra-chip outputs to both 0 and 1.
2. Where multiple VICs are daisy-chained in a system, read the expected values back from the VICITIP1 and VICITIP2 registers of the next VIC. For the **nVICIRQ** and **nVICFIQ** outputs, a 1 on the output pin will be shown as a 1 in the internal registers of the next VIC.

Figure 4-2 shows details of the implementation of the output integration test harness.

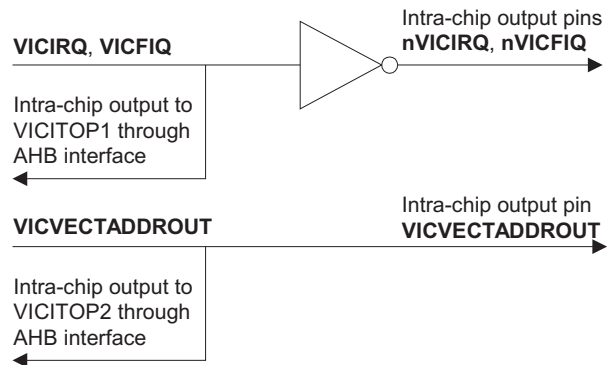


Figure 4-2 Output integration test harness

4.5 Integration test summary

Table 4-7 summarizes the integration test strategy for all PrimeCell VIC pins.

Table 4-7 PrimeCell VIC integration test strategy

Name	Type	Source/destination	Test strategy
HCLK	In	AHB	Register read/write
HRESETn	In	AHB	Register read/write
HADDR[11:2]	In	AHB	Register read/write
HSELVIC	In	AHB	Register read/write
HTRANS[1]	In	AHB	Register read/write
HWRITE	In	AHB	Register read/write
HWDATA[31:0]	In	AHB	Register read/write
HPROT[1]	In	AHB	Register read/write
HSIZE[2:0]	In	AHB	Register read/write
HREADYIN	In	AHB	Register read/write
HRDATA[31:0]	Out	AHB	Register read/write
HREADYOUT	Out	AHB	Register read/write
HRESP[1:0]	Out	AHB	Register read/write
VICINTSOURCE	In	Intra-chip	Use VICSoftInt register
nVICIRQIN	In	Intra-chip	Use VICITIP1 register
nVICFIQIN	In	Intra-chip	Use VICITIP1 register
VICVECTADDRIN	In	Intra-chip	Use VICITIP2 register
nVICIRQ	Out	Intra-chip	Use VICITOP1 register
nVICFIQ	Out	Intra-chip	Use VICITOP1 register
VICVECTADDRROUT	Out	Intra-chip	Use VICITOP2 register
SCANENABLE	In	Test controller	Not tested using integration test vectors
SCANINHCLK	In	Test controller	Not tested using integration test vectors
SCANOUTHCLK	Out	Test controller	Not tested using integration test vectors

Appendix A

ARM PrimeCell Vectored Interrupt Controller (PL190) Signal Descriptions

This appendix describes the signals that interface with the ARM PrimeCell Vectored Interrupt Controller (PL190). It contains the following sections:

- *AMBA AHB signals* on page A-2
- *Interrupt controller signals* on page A-4
- *Daisy chain signals* on page A-5
- *Scan test control signals* on page A-6.

A.1 AMBA AHB signals

The PrimeCell VIC module is connected to the AMBA AHB as a bus slave. Table A-1 shows the AHB signals that are used and produced.

Table A-1 AMBA AHB signal descriptions

Name	Type	Source/ destination	Description
HCLK	Input	Clock source	AMBA AHB bus clock, used to time all bus transfers. All signal timings are related to the rising edge of HCLK .
HRESETn	Input	Reset controller	AHB bus reset, active LOW.
HADDR[11:2]	Input	Master	System address bus.
HTRANS	Input	Master	Transfer type, which can be NONSEQUENTIAL, SEQUENTIAL, IDLE, or BUSY. This signal must be connected to HTRANS[1] on the AHB interface. HTRANS[0] is not used.
HWRITE	Input	Master	Transfer direction. Indicates a write transfer when HIGH, and a read transfer when LOW.
HSIZE[2:0]	Input	Master	Size of the transfer, which must be word (32-bit) for the VIC (HSIZE[2:0] = 0b010).
HPROT	Input	Master	Memory access protection type, which can be User mode (0) or privileged mode (1). This signal must be connected to HPROT[1] on the AHB interface. HPROT[3] , HPROT[2] and HPROT[0] are not used.
HWDATA[31:0]	Input	Master	Write data bus, used to transfer data from bus master to bus slaves during write operations.
HSELVIC	Input	Decoder	Slave select signal, which is a combinatorial decode of the address bus. It indicates that the current transfer is intended for the selected slave.

Table A-1 AMBA AHB signal descriptions (continued)

Name	Type	Source/ destination	Description
HRDATA[31:0]	Output	Slave	Read data bus, used to transfer data from bus slaves to bus master during read operations.
HREADYIN	Input	External slave	Transfer done signal, generated by an alternate slave. When HIGH, indicates that a transfer is complete. Can be driven LOW to extend a transfer.
HREADYOUT	Output	Slave	Transfer done signal, generated by the VIC. When HIGH, indicates that a transfer is complete. Can be driven LOW to extend a transfer.
HRESP[1:0]	Input	Slave	Transfer response, which provides additional transfer status information. The response can be OKAY, ERROR, RETRY or SPLIT. The PrimeCell VIC responds with either OKAY or ERROR.

A.2 Interrupt controller signals

Table A-2 shows the signals for the PrimeCell VIC that interface to the processor interrupt sources.

Table A-2 Interrupt controller signals

Name	Type	Source/ destination	Description
VICINTSOURCE [31:0]	Input	Peripheral interrupt request	Interrupt source input
nVICIRQ	Output	Interrupt controller	Interrupt request to processor
nVICFIQ	Output	Interrupt controller	Fast interrupt request to processor

A.3 Daisy chain signals

The daisy chain signals are used when two or more VICs are daisy-chained (see *Daisy-chained interrupt controller* on page 2-13). Table A-3 shows the daisy chain signals.

Table A-3 Daisy chain signals

Name	Type	Source/ destination	Description
VICVECTADDRIN [31:0]	Input	External interrupt controller	Connects to the VICVECTADDRROUT[31:0] signal of the previous VIC if daisy chaining is used. Connects to logic 0 if the VIC is not daisy-chained.
VICVECTADDRROUT [31:0]	Output	Interrupt controller	Connects to the VICVECTADDRIN[31:0] signal of the next VIC if daisy chaining is used. Left unconnected if the VIC is not daisy-chained.
nVICIRQIN	Input	External interrupt controller	Connects to the nVICIRQ signal of the previous VIC if daisy chaining is used. Connects to logic 1 if the VIC is the last in the daisy chain, or if VIC is not daisy-chained.
nVICFIQIN	Input	External interrupt controller	Connects to the nVICFIQ signal of the previous VIC if daisy chaining is used. Connects to logic 1 if the VIC is the last in the daisy chain, or if VIC is not daisy-chained.

A.4 Scan test control signals

The internal scan test control signals are shown in Table A-4.

Table A-4 Scan test control signals

Name	Type	Source/ destination	Description
SCANENABLE	Input	Scan controller	Scan enable
SCANINHCLK	Input	Scan controller	Scan data input for HCLK domain
SCANOUTHCLK	Output	Scan controller	Scan data output for HCLK domain

Appendix B

ARM PrimeCell Vectored Interrupt Controller (PL190) Example Code

This appendix provides examples of the code required when setting up the ARM PrimeCell Vectored Interrupt Controller (PL190). It contains the following section:

- *About the example code on page B-2.*

B.1 About the example code

The following examples of code are provided in this section:

- *Enable interrupts*
- *Disable interrupts*
- *Interrupt polling on page B-3*
- *Generate software interrupt on page B-3*
- *Clear software interrupt on page B-3*
- *FIQ interrupt initialization on page B-3*
- *FIQ interrupt handler on page B-4*
- *Simple interrupt initialization on page B-4*
- *Simple interrupt service routine on page B-5*
- *Vectored interrupt initialization on page B-5*
- *Vectored interrupt service routine on page B-6*
- *Daisy-chained vectored interrupt service routine on page B-6*
- *Highest level vectored IRQ interrupt service routine on page B-7.*

B.1.1 Enable interrupts

See Example B-1 for an example of the enable interrupt code.

Example B-1 Enable interrupts

LDR	r0, =IntCntlBase	;(where IntCntlBase is a ;predefined constant, e.g. ;IntCntlBase EQU 0xFFFFF000)
MOV	r1, #<interrupt to enable>	
STR	r1, [r0, #IntEnableOffset]	

B.1.2 Disable interrupts

See Example B-2 for an example of the disable interrupt code.

Example B-2 Disable interrupts

LDR	r0, =IntCntlBase
MOV	r1, #<interrupt to disable>
STR	r1, [r0, #IntEnableClearOffset]

B.1.3 Interrupt polling

See Example B-3 for an example of the interrupt polling code.

Example B-3 Interrupt polling

```

Loop    LDR    r0, =IntCntlBase
        LDR    r1, [r0, #RawInterruptOffset]
        CMP    r1, #0
        BEQ    loop
        Scan r1 for source of interrupt & branch to relevant routine

```

B.1.4 Generate software interrupt

See Example B-4 for an example of the generate software interrupt code.

Example B-4 Generate software interrupt

```

;Generate software interrupt on interrupt request line 1.
LDR    r0, =IntCntlBase
MOV     r1, #2 ;Interrupt source/request 1
STR     r1, [r0, #SoftIntOffset]

```

B.1.5 Clear software interrupt

See Example B-5 for an example of the clear software interrupt code.

Example B-5 Clear software interrupt

```

;Clear software interrupt on interrupt request line 1.
LDR    r0, =IntCntlBase                ;(where IntCntlBase is a
                                        ;predefined constant, e.g.
                                        ;IntCntlBase EQU 0xFFFFF000)

MOV     r1, #2
STR     r1, [r0, #SoftIntClearOffset]

```

B.1.6 FIQ interrupt initialization

See Example B-6 for an example of the FIQ interrupt initialization code.

Example B-6 FIQ interrupt initialization

```

LDR    r0, =IntCntlBase
MOV    r1, #<interrupt_to_enable>
STR    r1, [r0, #IntSelectOffset]      ;Select FIQ interrupt and clear
                                         ;other FIQs

STR    r1, [r0, #IntEnableOffset]      ;Enable interrupt

MRS    CPSR_c, #(DISABLE_IRQ + MODE_SYS_32) ;Enable FIQ interrupts

```

B.1.7 FIQ interrupt handler

See Example B-7 for an example of the FIQ interrupt handler code.

Example B-7 FIQ interrupt handler

```

;IRQ and FIQ interrupts are automatically masked until return from
;interrupt performed.
0x1c  Interrupt service routine
      Clear interrupt request
SUBS   pc, r14, #4

```

B.1.8 Simple interrupt initialization

Example B-8 shows how you can use the interrupt controller without using vectored interrupts or the interrupt priority hardware. For example, you can use it for debugging.

Example B-8 Simple interrupt initialization

```

LDR    r0, =IntCntlBase
MOV    r1, #<interrupt_to_enable>
LDR    r2, [r0, #IntSelectOffset]      ;Select IRQ interrupt
BIC    r2, r2, r1
STR    r2, [r0, #IntSelectOffset]
STR    r1, [r0, #IntEnableOffset]      ;Enable interrupt

MRS    CPSR_c, #(DISABLE_IRQ + MODE_SYS_32) ;Enable FIQ interrupts

```

B.1.9 Simple interrupt service routine

Example B-9 shows how you can use the interrupt controller without using vectored interrupts or the interrupt priority hardware. For example, you can use it for debugging.

Example B-9 Simple interrupt service routine

```

;This interrupt service routine assumes that there are no vectored
;interrupts. It also assumes that interrupts are disabled until the
;interrupt service routine has been exited.

;IRQ interrupts are masked until a return from interrupt is performed. The
;FIQ interrupt is enabled.

0x18  B      IRQ_ISR                      ;Branch to interrupt service routine

IRQ_ISR
    STMFD    sp!, {r0, r1}                ;Store r0 and r1
    LDR      r0, [IntCntlBase]
    LDR      r1, [r0, #IRQStatusOffset]    ;Discover source of interrupt
    Scan r1 for source of interrupt & branch to relevant routine ISR
    Interrupt service routine
    Clear interrupt request
    LDMFD    sp!, {r0, r1}                ;Restore r0 and r1
    SUBS     pc, r14, #4                   ;Exit from IRQ

```

B.1.10 Vectored interrupt initialization

See Example B-10 for an example of the vectored interrupt initialization code.

Example B-10 Vectored interrupt initialization

```

LDR    r0, =IntCntlBase
MOV    r1, #<interrupt_to_enable>

STR    r1, [r0, #IntEnableClearOffset] ;Disable interrupt

LDR    r2, =default_vector_address      ;Set default vector address
STR    r2, [r0, #DefaultVectorAddressOffset]
    ;Setup and enable vectored interrupt 15
MOV    r2, #vector_address              ;Set vector address
STR    r2, [r0, #VectorAddr150ffset]
MOV    r2, #interrupt_source            ;Set interrupt source

```

```

    ORR    r2, r2, #0x20                ;and enabled vector interrupt
    STR    r2, [r0, #VectorCntl150ffset]

    LDR    r2, [r0, #IntSelectOffset]    ;Select IRQ interrupt
    BIC    r2, r2, r1
    STR    r2, [r0, #IntSelectOffset]
    STR    r1, [r0, #IntEnableOffset]    ;Enable interrupt

    MRS    CPSR_c, #(DISABLE_IRQ + MODE_SYS_32) ;Enable FIQ interrupts

```

B.1.11 Vectored interrupt service routine

See Example B-11 for an example of the vectored interrupt service routine code.

Example B-11 Vectored interrupt service routine

```

0x18    LDR    pc, [pc, #-0xff0]        ;Load Vector into PC

vector_handler
    ;Code to enable interrupt nesting
    STMFD    sp!, {r12,r14}            ;Stack workspace
    MRS      r12, spsr                 ;Save SPSR into r12
    MSR      cpsr_c, #0x1F             ;Reenable IRQ, go to system mode

    Interrupt_service_routine

    ;Code to exit handler
    MSR      cpsr_c, #0x52             ;Disable IRQ, move to IRQ mode
    MSR      spsr, r12                 ;Restore SPSR from r12
    LDMFD    sp!, {r12,r14}            ;Restore registers
    STR      r0, VectorAddr             ;Acknowledge Vectored IRQ has
                                        ;finished
SUBS      pc, r14, #4                  ;Return from IRQ

```

B.1.12 Daisy-chained vectored interrupt service routine

See Example B-12 for an example of the daisy-chained vectored interrupt service routine code.

Example B-12 Daisy-chained vectored interrupt service routine

```

0x18    LDR    pc, [pc, #-0xff0]        ;Load vector into PC

```

```

daisy_vector_handler
    ;Code to enable interrupt nesting
    STMFD sp!, {r12,r14}                ;Stack workspace
    LDR    r12, VectorAddrDaisyVIC      ;Read VectorAddrDaisyVIC to
                                        ;ensure hardware priority logic is
                                        ;enabled correctly

    LDR    r12,[r12]
    MRS    r12, spsr                    ;Save SPSR into r12
    MSR    cpsr_c, #0x1F                ;Reenable IRQ, go to system mode

    Interrupt_service_routine

    ;Code to exit handler
    MSR    cpsr_c, #0x52                ;Disable IRQ, move to IRQ mode
    MSR    spsr, r12                    ;Restore SPSR from r12
    LDMFD  sp!, {r12,r14}                ;Restore registers
    STR    r0, VectorAddrDaisyVIC
    SUBS   pc, r14, #4                  ;Return from IRQ

```

B.1.13 Highest level vectored IRQ interrupt service routine

See Example B-13 for an example of the highest level vectored IRQ interrupt service routine code.

Example B-13 Highest level vectored IRQ interrupt service routine

```

0x18  LDR    pc, [pc, #-0xff0]          ;Load vector into PC

highest_priority_vector_handler

    Interrupt_service_routine

    ;Code to exit handler
    STR    r0, VectorAddr                ;Acknowledge Vectored IRQ has
                                        ;finished
    SUBS   pc, r14, #4                  ;Return from IRQ

```

Index

The items in this index are listed in alphabetical order, with symbols and numerics appearing at the end. The references given are to page numbers.

A

AMBA 1-2

D

Daisy-chain 2-6, 2-12, A-5

F

Fast interrupt request 2-2

FIQ 2-2

I

Integration testing
 block outputs 4-10
Interrupt flow sequence
 standard 2-10
 vectored 2-10

Interrupt latency 1-2, 2-2, 2-12
Interrupt masking 2-9
Interrupt priority 2-2
Interrupt priority logic 2-8
Interrupt priority register 2-10
Interrupt request 2-2, 2-5
Interrupt service routine 2-9, 2-10
IRQ 2-2, 2-10
ISR 2-10

N

Nonvectored interrupt 2-2, 2-6

S

Scan testing 4-3
SoC 1-2
Software interrupt 2-2, 2-9, 2-10
Standalone 2-12

T

Test harness 4-2

V

Vectored interrupt 2-2, 2-9
Vectored Interrupt Controller 1-2, 2-2
VIC registers 3-3
 VICDefVectAddr 3-10
 VICFIQStatus 3-7
 VICIntEnable 3-8
 VICIntEnClear 3-8
 VICIntSelect 3-7
 VICIRQStatus 3-6
 VICITCR 3-12, 4-4
 VICITIP1 3-12, 4-5
 VICITIP2 3-12, 4-5
 VICITOP1 3-13, 4-6
 VICITOP2 3-13, 4-6
 VICPCellID 3-17
 VICPeriphID 3-14

VICProtection 3-9
VICRawIntr 3-7
VICSoftInt 3-8
VICSoftIntClear 3-9
VICVectAddr 3-10, 3-11
VICVectCntl 3-11